GenerativeUI 아침 보고서, 2026-11-08

generativeui 논문 에이전트, WebSight 결과 반영본

독립 test 100개와 외부 데이터 WebSight 50개 결과 반영.

1. 한 장 요약

핵심 결과 5개

1. **구조 규칙은 실제 GPU 시간에서 prompt lookup보다 약 7% 빠르고, HTML 표현을 바꿔도 대체로 유지된다.** 원래 형식 +6.95% [+4.06, +10.00], 요청 50개 × 3회. 재실행 +6.76% [+3.73, +9.89]. 예시와 출력 형식을 모두 바꾼 조건 A에서는 네 변형 모두 유의하다: 서식 +6.4% [+0.4, +12.7], CSS 인라인 +11.9% [+6.4, +16.8], 래퍼 추가 +6.9% [+4.2, +9.6], 클래스 이름 +6.2% [+1.2, +10.7]. 초안 원천만 바꾼 조건 B에서는 초 단위로 셋이 유의하고, 래퍼 추가는 +0.18초 [−0.03, +0.37]로 유의하지 않다. **단, 독립 test 100개에서는 규칙 단독 이득이 +2.3% [−2.9, +6.7]로 유의하지 않다.** 호출당 토큰은 +10% 높지만, 48개 요청에서 출력이 수치 차이로 갈라져 규칙 쪽 출력이 평균 34토큰 길었다. **다른 HTML 출처인 WebSight 50개에서도 규칙 이득은 남는다.** 요청당 +1.51초 [+0.14, +2.81], 호출 +4.17% [+0.71, +7.38]로 유의하다. 요청별 비율로는 +3.22% [−0.24, +6.46]로 0을 조금 포함한다.
2. **학습 순위 모델은 원래 형식에서 규칙보다 실측 5.1% 빠르지만, 형식이 바뀌면 이득이 사라지거나 뒤집힌다.** 원래 형식 +5.1% [+1.8, +8.1], 초 단위 +0.29초 [+0.05, +0.53]. 조건 A 서식 변환에서 −11.5% [−27.8, −1.1]로 더 느리고, 나머지 세 변환은 유의하지 않다. 조건 B에서는 클래스 변환만 유의하다, +0.25초 [+0.07, +0.45]. **외부 데이터 WebSight에서는 규칙 대비 이득이 없다.** 규칙 − 학습 −0.53% [−3.35, +2.22]. lookup 대비 +3.50% [+0.85, +5.97]는 규칙의 몫과 같은 크기다.
3. **정식 SuffixDecoding은 우리 방법 단독보다 훨씬 강하다. DOM 특징을 더한 추가 이득은 1–3%로 작다.** 재생 분석, 주 데이터 114개, 요청당 모델 호출 수: 학습 모델 단독 121.8, 공식 규칙 SD 기본 설정 108.1, 튜닝 94.3. SD에 학습 순위 모델을 더한 이득은 튜닝 설정에서 2.9 [2.25, 3.61], 즉 −3.1%이고, 초안 상한 64에서는 0.96 [0.40, 1.54]이다. 공식 기본 설정에서는 주 데이터에서 0.52 [0.14, 0.89]이고, 데이터 세트 셋 중 둘에서는 유의하지 않다. DOM 특징만의 몫은 0.96–1.70 호출이다. GPU 실측은 아직 없다.
4. **SD의 전역 캐시 이득은 누수가 아니다.** 평가 요청 174개 모두 같은 요청, 같은 앱, 정답과 똑같은 출력이 캐시에 0개였다. 실제 서빙 순서로 캐시를 쌓아도 prompt lookup 대비 이득의 79%가, 상용구를 지워도 82%가 남는다.
5. **처음 보는 앱에서도 학습 모델이 규칙과 표준보다 실측으로 빠르고, 품질은 같다.** train 앱 150개로만 학습한 모델을 pilot과 겹치지 않는 test 100개 요청, 앱 92개에서 1회 실측했다. 시간은 lookup 5.28초, 규칙 5.17초, 학습 모델 4.86초. 규칙 − 학습 +5.9% [+3.2, +8.7], 요청당 +0.31초 [+0.08, +0.58]. lookup − 학습 +9.5% [+5.0, +13.3]. 체크리스트 0.94, 요소 F1 0.59로 정책 간 차이는 ±0.01 이내이고 유의하지 않다. 단, 이것은 같은 S2W 변환기 형식 안의 일반화다. 다른 출처인 WebSight에서는 학습 모델이 규칙과 같다.

최종 권고: Conditional Go

PLAN.md의 방향 유지 조건 네 가지로 판단했다.

	조건	판정	근거
1	GPU 실제 시간 개선	충족	pilot에서 규칙 +6.95%, 학습 모델은 규칙 대비 +5.1%. 독립 test에서 학습 모델은 규칙 대비 +5.9%, lookup 대비 +9.5%. 규칙 단독은 test에서 유의하지 않음. WebSight에서 규칙은 lookup보다 요청당 +1.51초
2	구조 특징의 독립 기여	충족, 재생 분석	구조 특징 제거 시 +3.3 [3.0, 3.7] 호출. SD 위에서도 DOM 몫 0.96–1.70
3	표현 변화와 다른 출처에 강건	규칙만 충족	규칙은 조건 A 네 변형과 WebSight에서 유의. 학습 모델은 서식 변환에서 −11.5%, WebSight에서 규칙 대비 −0.5%로 유의하지 않음
4	정식 SD 대비 추가 효용	약하게 충족	튜닝 설정에서만 1–3%. 기본 설정은 0.5% 또는 유의하지 않음. GPU 실측 없음

- **주 기여:** 구조 규칙은 다른 HTML 출처에서도 lookup보다 빠르다. 학습된 구조 조건 초안 선택은 같은 HTML 출처의 처음 보는 앱에서 규칙보다 5.9%, 표준보다 9.5% 빠르고 품질을 유지한다. 다른 출처인 WebSight에서는 학습 모델의 추가 이득이 없다.

- **충족된 조건:** 독립 test에서 학습 모델이 규칙보다 느리지 않을 것. 충족했다.
- **남은 조건 두 가지:** SD + DOM의 GPU 실측 이득 확인, 그리고 서식 변화와 다른 출처에서 학습 모델의 강건성 회복. 실패하면 SD 위의 추가 이득과 형식 및 출처 일반화는 주장에서 뺀다.
- **다음 연구 후보:** 전역 캐시까지 포함한 위치 검색. 로컬 문맥 오라클 74.5 대비 전역 포함 오라클 46.9 호출로, SD 튜닝 94.3과의 격차가 가장 크다.
- **멈출 것:** "구조 정보만으로 SuffixDecoding을 이긴다"는 주장, 형식 변화나 다른 HTML 출처에서 학습 모델이 강건하다는 주장.

2. 완료된 실험의 결과표

공통 설정: Gemma 4 E2B, bitsandbytes 4비트, 단일 RTX 4070 12GB, 배치 1, greedy, 정확 검증, 질의 query_mid, 예시는 TF-IDF 상위 3개. 신뢰구간은 요청 단위 짝 부트스트랩 95%.

2.1 기존 기술 조합의 기준선, 요청 10개 중앙값

방법	전체 시간	화면 절반	배수	체크리스트
예시 없음, 일반 생성	59.4초	48.2초	1.0	0.867
예시 3개, 일반 생성	27.1초	20.3초	2.2	1.000
예시 3개, prompt lookup	6.6초	4.5초	9.0	1.000
구조 예시, 화면 기준 수락	5.3초	3.3초	11.2	1.000

11배는 기존 기술의 성과다. 출처 `runs/poc/E2B_Ctable.md`, `E2B_perc.md`.

2.2 GPU 실측: 초안 위치 선택, 요청 50개 × 3회

정책	시간	호출	토큰/호출	초안 비용 비중
prompt lookup	6.53초	96.9	4.98	0.23%
구조 규칙	6.12초	90.0	5.52	0.64%
학습 순위 모델	5.83초	85.5	5.85	0.85%
lookup − 규칙: +0.370초 [+0.181, +0.572], +6.95% [+4.06, +10.00] (gate2_p50)				
lookup − 규칙, 재현: +0.410초 [+0.204, +0.623], +6.76% [+3.73, +9.89] (gate2b)				
규칙 − 학습: +0.293초 [+0.049, +0.531], +5.13% [+1.83, +8.10]				
lookup − 학습: +0.702초 [+0.434, +0.980], +11.87% [+8.49, +15.24]				

재생 분석과 실측 호출 수는 450/450 일치했다.

2.3 표현 변화 강건성 P1, 요청 50개 1회

조건 A: 예시와 초안 원천 모두 변형. 출처 `POC_method.md` §6.4, `runs/poc/p1A`.

변형	lookup / 규칙 / 학습	lookup − 규칙	규칙 − 학습
원래 형식	6.53 / 6.12 / 5.83초	+6.8% [+3.7, +9.9]	+5.1% [+1.8, +8.1]
서식	17.2 / 15.4 / 16.1초	+6.4% [+0.4, +12.7]	−11.5% [−27.8, −1.1]
CSS 인라인	9.46 / 8.17 / 8.07초	+11.9% [+6.4, +16.8]	0.0% [−6.9, +5.3]
래퍼 추가	7.07 / 6.57 / 6.34초	+6.9% [+4.2, +9.6]	+3.7% [−0.9, +8.3]
클래스 이름	7.04 / 6.46 / 6.33초	+6.2% [+1.2, +10.7]	+2.6% [−2.1, +6.6]

조건 B: 초안 원천만 변형, GPU 실측, 초 단위. 출처 `runs/poc/p1B/<변형>/gate2_summary.md`.

변형	lookup / 규칙 / 학습	lookup − 규칙	규칙 − 학습
서식	8.72 / 7.94 / 7.75초	+0.779 [+0.321, +1.463]	+0.185 [−0.006, +0.378]
CSS 인라인	11.98 / 11.39 / 11.48초	+0.587 [+0.283, +0.897]	−0.084 [−0.481, +0.262]
래퍼 추가	6.07 / 5.88 / 5.66초	+0.183 [−0.031, +0.367]	+0.227 [−0.360, +0.636]
클래스 이름	6.40 / 6.09 / 5.83초	+0.317 [+0.163, +0.498]	+0.251 [+0.068, +0.450]

- 조건 B 요약 파일에는 초 단위 신뢰구간과 요청별 비율 신뢰구간이 함께 있다. 래퍼 추가의 lookup − 규칙 +3.48% [+0.61, +5.94], 서식과 래퍼의 규칙 − 학습 +2.52% [+0.67, +4.31], +5.65% [+0.51, +9.69]는 비율 기준으로만 유의하다. 이 보고서는 초 단위로 판정했다.
- 품질: 조건 A 체크리스트는 모든 변형과 정책에서 0.89–0.96이고 원래는 약 0.92다. 체계적 품질 손실은 없다.

2.4 재생 분석: 학습 순위 모델

항목, 요청당 호출	값 [신뢰구간]
규칙 − 학습, 444개 출력, 앱 단위 cross-fit	6.6 [4.2, 10.0]
구조 특징 제거 시 손실	+3.3 [3.0, 3.7]
이전 초안 이력 특징 제거 시 손실	+0.1 [−0.3, 0.6]
규칙 − train-split 모델, 전체	3.8 [2.9, 4.9]
규칙 − train-split 모델, GPU 실측 출력 74개	3.9 [3.4, 4.3]

2.5 재생 분석: 정식 SuffixDecoding, 주 데이터 114개, 초안 상한 16, 공식 규칙

방법	호출	SD − (SD + 학습)	상한 16	상한 64
prompt lookup	134.2	기본 설정	0.52 [0.14, 0.89]	0.19 [−0.09, 0.47]
구조 규칙 / 학습 모델	126.0 / 121.8	튜닝	2.9 [2.25, 3.61]	0.96 [0.40, 1.54]
SD 로컬 문맥만	153.0			
SD 기본 / + 학습	108.1 / 107.5			
SD 튜닝 / + 학습	94.3 / 91.4			
SD 후보 집합 오라클	80.5			
로컬 문맥 어디든 오라클	74.5			
로컬 + 전역 캐시 오라클	46.9			

- 표현 변화에서 SD + DOM 추가 이득(튜닝): 원래 2.10 [1.22, 3.06], CSS 인라인 6.04 [4.16, 8.18], 래퍼 4.10 [2.64, 5.64]. DOM 특징만의 몫은 0.96, 1.38, 1.70이고 나머지는 SD 후보를 학습으로 재순위화한 효과다.
- SD의 lookup 대비 이득은 형식을 바꿔도 줄지 않았다. 원래 19.7%, 조건 B 서식 39.8%, CSS 57.9%. 조건 A 래퍼 추가에서만 14.6%로 줄었다.
- 캐시 감사: 누수 0건. 서빙 순서 캐시 103.0 호출, cross-fit 94.9보다 +8.2 [5.6, 11.4]. 상용구 제거 캐시 102.0.
- 공식 구현과의 차이: 알고리즘은 같다. 전역 트리에 요청 자신의 응답을 넣는 규칙은 반영했다. 초안 상한은 공식 vLLM 기본 16, 독립 시뮬레이터 기본 64다.

2.6 예시 선택 실험, 요청 50개

비교	값 [신뢰구간]
구조 예시 − TF-IDF 예시, 체크리스트	+3.3%p [+0.2, +7.2]
같은 비교, 시간	+0.15초 [−0.51, +0.79]
코드 겹침 최대 예시 − TF-IDF, 시간	+1.40초 [+0.41, +2.42]
6개 후보 중 최속 − TF-IDF 1위, 시간	−4.60초 [−6.21, −3.11], −37%
학습한 예시 선택기 − TF-IDF 1위	−0.59초 [−2.61, +1.33], 격차의 13%

2.7 독립 test와 외부 데이터 WebSight

E(ii) 독립 test: 요청 100개, 앱 92개, pilot과 겹치지 않음, train-split 학습 모델, 1회. 출처 `runs/poc/E/test100/gate2_summary.md`, `quality.md`.

정책	시간	호출	토큰/호출	체크리스트	요소 F1
prompt lookup	5.28초	88.5	5.20	0.944	0.591
구조 규칙	5.17초	86.0	5.74	0.938	0.586
학습 순위 모델	4.86초	80.5	6.22	0.943	0.587

규칙 − 학습: +0.310초 [+0.083, +0.581], +5.90% [+3.15, +8.72]
lookup − 학습: +0.419초 [+0.149, +0.655], +9.51% [+5.03, +13.29]
lookup − 규칙: +0.109초 [−0.179, +0.352], +2.30% [−2.93, +6.70], 유의하지 않음
품질 짝 차이는 모두 ±0.01 이내, 유의하지 않음

F WebSight 50개: WebSight v0.2 eval, S2W와 다른 HTML 출처, train-split 학습 모델, 1회. 출처 `runs/poc/F_websight/gate2_summary.md`, `websight_quality.md`.

정책	시간	호출	토큰/호출	출력 토큰	요소 F1	렌더 성공
prompt lookup	39.79초	683.9	2.06	1420	0.454	1.0
구조 규칙	38.28초	651.3	2.16	1412	0.467	1.0
학습 순위 모델	38.28초	649.5	2.15	1402	0.469	1.0
lookup − 규칙: +1.512초 [+0.143, +2.805], 호출 +4.17% [+0.71, +7.38], 요청별 비율 +3.22% [−0.24, +6.46]						
규칙 − 학습: −0.005초 [−1.073, +1.089], −0.53% [−3.35, +2.22], 유의하지 않음						
lookup − 학습: +1.507초 [+0.491, +2.499], +3.50% [+0.85, +5.97]						

- 규칙 이득은 다른 출처로 옮겨간다. 초 단위와 호출 수로 유의하다. 요청별 비율은 0을 조금 포함하므로 조건 B와 같이 초 단위로 판정했다.
- 학습 모델의 이득은 옮겨가지 않는다. lookup 대비 이득은 규칙의 몫과 같은 크기다.
- 품질은 같다. 요소 F1은 0.454–0.469이고 50개 모두 렌더링된다.
- 해석할 때 주의할 점이 있다. 호출당 토큰이 2.1개로 S2W의 5–6개보다 훨씬 적어 초안이 잘 맞지 않는다. 출력은 평균 약 1,410토큰으로 정답 평균 525토큰보다 훨씬 길고, 2,048토큰 상한에 닿은 요청이 lookup 5개, 규칙 2개, 학습 4개다. 정책 사이 출력이 같은 요청은 1–2개뿐이라 출력 길이 차이가 시간 차이에 섞인다.
- 정답 대리 재생은 규칙의 호출 감소를 6.1%로 예측했다. 실측은 4.2%로 더 작지만 방향은 같다.

3. 확인된 사실과 가설

확인된 사실, 신뢰구간 > 0

- 구조 규칙은 GPU 실측에서 prompt lookup보다 빠르고, 조건 A 네 변형에서도 유지된다.
- 학습 모델은 원래 형식에서 규칙보다 GPU 실측으로 빠르다. 처음 보는 test 앱 100개 요청에서도 규칙보다 +5.9%, lookup보다 +9.5% 빠르고 품질이 같다.
- 다만 독립 test에서 규칙 단독의 lookup 대비 이득은 +2.3%로 유의하지 않다.
- 다른 HTML 출처인 WebSight에서도 규칙은 lookup보다 요청당 +1.51초, 호출 +4.2%로 유의하게 빠르다.
- WebSight에서 학습 모델은 규칙보다 빠르지 않다. 차이는 −0.5%이고 유의하지 않다. 학습 모델의 이득은 S2W 변환기 형식 안에서만 확인됐다.
- 구조 특징을 빼면 학습 모델이 나빠진다. 재생 분석이다.
- 정식 SD 튜닝 설정 위에 학습 모델을 더하면 호출이 1–3% 준다. 재생 분석이다.
- SD의 전역 캐시 이득은 누수나 상용구로 설명되지 않는다.
- 정확 검증에서는 한 단계 최선 선택이 다단계 계획과 같다. 70개 출력 모두 차이 0이다.

가설, 아직 검증하지 않음

- SD + DOM의 1–3% 이득이 GPU 시간에서도 남는다. 튜닝 설정은 호출당 검증 토큰이 13.5개라 시간 이득이 줄 수 있다.
- 전역 캐시를 포함한 위치 검색이 오라클 격차 94.3 → 46.9의 일부를 메울 수 있다.
- 형식이 섞인 데이터로 학습하거나 형식 의존 특징을 정규화하면 서식 변환 실패를 막을 수 있다.
- 외부 출처 데이터를 섞어 학습하면 WebSight에서도 학습 모델의 이득이 생긴다.
- WebSight에서 규칙 이득이 작은 것은 출력이 정답보다 길고 예시와 덜 겹쳐 초안이 덜 맞기 때문이다.

4. 실패 사례와 원인

- **서식 변환에서 학습 모델이 규칙보다 느리다.** 모델이 예시에 없는 방식으로 `<style>` 안 CSS까지 줄바꿈한다. 예시 CSS에서 가져온 초안이 맞지 않아 평균 수락 길이가 3.99에서 1.51로 떨어진다. 깊이, 형제 순번, 최근 수락 길이가 학습 범위를 벗어나 MLP가 잘못 보정된다. 초안 원천만 바뀐 조건 B에서는 실패하지 않으므로 출력 쪽 분포 이동이 원인이다.
- **SD의 전역 캐시 우위.** 우리 방법은 요청 안의 문맥만 쓰고 SD는 다른 요청의 출력을 쓴다. 이전의 "변환기 형식 공유가 SD를 유리하게 했을 것"이라는 해석은 반박됐다. 형식을 바꿔도 SD 이득은 래퍼 추가를 빼면 유지되거나 커졌고, 상용구를 지워도 82%가 남았다.

- **공식 기본 설정에서 SD + DOM 이득 소멸.** 초안이 짧거나 길 때 SD의 빈도 기반 선택이 이미 잘 맞는다.
- **빠른 예시를 미리 고르는 학습 실패.** 시간 차이의 75%가 출력 길이에서 오고, 생성 전 특징의 상관은 모두 0.13 이하다.
- **화면 기준 수락의 여지가 작다.** 거절된 초안 중 같은 화면을 만드는 것은 6–18%뿐이다.
- **WebSight에서 학습 모델의 이득이 사라진다.** 규칙 − 학습은 −0.53% [−3.35, +2.22]다. 학습 데이터가 S2W 변환기 형식뿐이다. 특징 분포를 비교하는 원인 분석은 아직 하지 않았다.

5. 다음 작업 우선순위

1. 독립 test에서 규칙과 lookup의 차이를 3회 반복 또는 같은 출력 기준 재생으로 다시 확인한다.
2. SD + DOM의 GPU 실측. 공식 모듈 빌드 승인이 필요할 수 있다.
3. 학습 모델 개선: 형식 혼합 학습, WebSight pool 150개를 섞은 학습, 형식 의존 특징 정규화, 이력 특징 제거.
4. 새 방향: 전역 캐시를 포함한 위치 검색.

6. 사용자 결정이 필요한 사항

- **공식 SuffixDecoding C++ 모듈 빌드 승인.** 격리된 폴더에서 외부 코드를 빌드하고 실행해야 하는데 권한 정책에 막혔다. 승인하면 공식 구현으로 재생 분석을 교차 검증할 수 있다. vLLM 서빙 경로는 Windows에서 동작하지 않는다.
- **코드 라이선스.** 공식 SuffixDecoding은 Apache-2.0이다. 우리 코드는 공식 코드를 복사하지 않고 규칙만 다시 구현했다. 우리 저장소의 라이선스를 정해야 한다.
- **데이터 공개 범위.** 데이터는 Screen2Words와 RICO, CC-BY-4.0에서 파생했다. 파생 데이터, 모델 출력, 학습 가중치의 공개 범위를 정해야 한다.